

AbeAttributes

AbeAttributes 是一个基于 Unity Editor 的可扩展 Inspector Framework。

它将 Inspector 中不同职责拆分成独立的系统：

- Property Model
- Type / Member Scanning
- Accessor
- Method Invoker
- Processor
- Validator
- State Updater
- Drawer Locator
- Drawer Chain
- Drawer

每个系统负责不同阶段的工作，使 Property 的解析、处理、验证、状态更新和绘制互相独立。

Core Architecture

整体结构：

AbeInspector

|

▼

AbePropertyTree

|

▼

AbeProperty

|

├─ TypeScanner

├─ MemberScanner

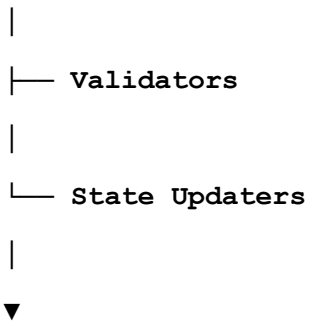
├─ Accessor

└─ Attribute Information

|

▼

Processors



DrawerLocator



DrawerChain



Drawers



Unity Editor GUI

可以将整个流程简化为：

Property



Scan / Resolve



Process



Validate



Update State



Find Drawers



Build Drawer Chain



Draw

Main Components

AbeInspector

AbeInspector 是 Unity Inspector 的入口。

它负责：

- 创建 Property Tree
- 更新 Property
- 执行 Inspector 绘制
- 管理 Inspector 生命周期

它本身不负责具体的 Attribute 逻辑。

Unity Editor

↓

AbeInspector

↓

AbePropertyTree

AbePropertyTree

AbePropertyTree 是整个 Inspector Property Model 的容器。

它负责：

- 管理 Root Properties
- 查找 Property
- 管理 Property 生命周期
- 执行 Property Processing
- 更新 Property State
- 执行 Property Drawing

结构：

AbePropertyTree

|

└─ AbeProperty

```
|— AbeProperty
|— AbeProperty
└— AbeProperty
```

Property 可以继续拥有 Children:

AbeProperty

```
|— Child
|— Child
└— Child
```

SerializedProperty Children 使用 Lazy Creation。

只有实际访问 Child 时才创建对应的 AbeProperty。

AbeProperty

AbeProperty 是 Inspector 中一个 Property 的核心 Model。

它保存一个 Property 在 Inspector 中需要的信息，例如：

AbeProperty

```
|
|— Name
|— Type
|— Parent
|— Attributes
|— SerializedProperty
|— State
└— Children
```

Property Model 的存在让后续系统不需要不断重新从 Unity 的 SerializedProperty 或 Reflection 中解析相同的信息。

TypeScanner

TypeScanner 负责分析一个 Type 的结构。

例如：

Type

```
|
```

- |— Fields
- |— Properties
- |— Methods
- |— Attributes

扫描结果可以被缓存和复用。

它主要解决：

“这个 Type 有什么？”

MemberScanner

MemberScanner 负责发现 Type 中可以被系统使用的 Members。

例如：

Field

Property

Method

它主要解决：

“这个 Type 里面有哪些 Member？”

之后这些 Member 可以交给：

Accessor

MethodInvoker

Member Resolver

继续处理。

Accessor

Accessor 用于读取或修改 Member 的 Value。

例如：

```
public int Health;
```

Accessor 可以提供：

Get → Health

Set → Health

它将 Reflection Member 转换成统一的数据访问方式。

因此其他系统不需要直接依赖：

`FieldInfo`

`PropertyInfo`

`MethodInvoker`

`MethodInvoker` 用于调用被解析出来的方法。

例如：

```
private bool IsEnabled()  
{  
    return true;  
}
```

处理流程：

`Method`

↓

`MemberScanner`

↓

`Resolver`

↓

`MethodInvoker`

↓

`Invoke()`

`Parameterless Method` 可以被转换成缓存 `Delegate`，从而避免 `Inspector` 每次 `Repaint` 都重新进行完整 `Reflection Invocation`。

`Attribute Resolution`

`Property` 上的 `Attribute` 会被系统解析并提供给后续处理流程。

例如：

`[Required]`

```
public string Name;
```

系统可以得到：

`AbeProperty`

↓

Attribute

↓

RequiredAttribute

Attribute 本身只是描述：

“这个 Property 被赋予了什么额外信息或行为？”

之后由不同系统决定这个 Attribute 应该如何处理。

Processors

Processor 负责处理 Property。

它主要解决：

“这个 Property 应该进行什么额外处理？”

Processor 可以根据：

- Property
- Type
- Attribute
- Metadata
- Context

执行对应逻辑。

Processor 本身不等于 Validator，也不等于 Drawer。

Processor

|

├─ Validator

├─ State Updater

└─ Other Processing

因此 Processor 更像是 Property Processing 的扩展入口。

Validators

Validator 专门负责：

检查 Property 或 Value 是否符合指定条件。

例如：

```
[Required]
```

```
public string Name;
```

Required Validator 可以检查：

Name

↓

是否为空？

↓

Validation Result

其他例子：

MinValue

MaxValue

RequiredType

ValidateInput

这些功能的核心目的都是：

检查

↓

得到 Validation Result

而不是绘制 GUI。

Validator 与 Drawer 的区别

这是系统中非常重要的区别。

例如：

```
[Required]
```

```
public string Name;
```

Required 的主要职责是：

Required

↓

Validator

↓

检查 Name

而不是:

Required

↓

Drawer

↓

绘制 Name

因此:

Validator

→ 检查数据

Drawer

→ 绘制数据

两者是不同的职责。

State Updaters

State Updater 负责根据当前 Property 状态更新 Inspector State。

例如:

[ReadOnly]

```
public int Health;
```

如果 ReadOnly 在当前系统中表示:

不能编辑

那么它可以通过 State Updater 转换成:

ReadOnly

↓

StateUpdater

↓

Enabled = false

最终 Drawer 只需要读取:

Property.State.Enabled

而不需要知道为什么它是 Disabled。

State Updater 可以处理什么？

例如：

Visible

Enabled

Expanded

状态更新流程：

Attribute / Condition

↓

State Updater

↓

Property State

↓

Drawer

例如：

```
[ShowIf(nameof(IsVisible))]
```

```
public int Value;
```

可以：

ShowIf

↓

StateUpdater

↓

IsVisible()

↓

Visible = true / false

当前 ShowIf 状态可以通过现有的：

```
PropertyUtility.IsVisible()
```

进行处理。

Processor / Validator / State Updater

这三个概念可以这样区分：

系统	主要问题	例子
Processor	Property 要进行什么处理？	Property Metadata / Attribute Processing
Validator	Property / Value 是否有效？	Required / MinValue / MaxValue
State Updater	Property 当前应该是什么状态？	Visible / Enabled / Expanded

简单来说：

Processor

→ Process

Validator

→ Validate

State Updater

→ Update State

它们都发生在 GUI Drawing 之前。

Drawer System

Drawer 专门负责：

“这个 Property 在 Inspector 里应该怎么画？”

Drawer 不负责验证数据，也不负责决定 Property 是否有效。

它接收已经处理过的 Property，然后负责 GUI。

Property

↓

Processing

↓

Validation

↓

State Update

↓

Drawer

↓

GUI

Drawer Types

AbeAttributes 提供三种主要 **Drawer**:

AbeAttributeDrawer<TAttribute>

AbeGroupDrawer<TAttribute>

AbeValueDrawer<TValue>

三种 **Drawer** 的匹配依据不同。

Attribute Drawer

→ **Attribute**

Group Drawer

→ **Group**

Value Drawer

→ **Value Type**

Attribute Drawer

AbeAttributeDrawer<TAttribute> 用于:

某个 **Attribute** 本身定义了特殊的 **Inspector** 绘制方式。

例如:

```
[Dropdown(...)]
```

```
public int Selected;
```

Dropdown 的意义是:

这个 **Property** 应该使用 **Dropdown GUI**

因此:

DropdownAttribute

↓

Dropdown Drawer

↓

Popup / Dropdown GUI

另一个例子：

```
[ProgressBar]
```

```
public float Health;
```

可以：

```
ProgressBarAttribute
```

↓

```
ProgressBar Drawer
```

↓

```
Progress Bar GUI
```

所以 Attribute Drawer 的判断依据是：

```
Property
```

↓

有没有这个 Attribute？

而不是 Property 的 Value Type。

Group Drawer

AbeGroupDrawer<TAttribute> 用于：

绘制一组 Property 的整体结构。

例如：

```
[BoxGroup("Combat")]
```

```
public int Health;
```

```
[BoxGroup("Combat")]
```

```
public int Damage;
```

最终：

```
|-----|
|  Combat  |
|          |
```

Health	100
Damage	25

这里 Drawer 关心的不是：

Health 是 int

Damage 是 int

而是：

Health

Damage

↓

属于 Combat Group

因此使用：

`AbeGroupDrawer<BoxGroupAttribute>`

Group Drawer 可以控制：

- Group Header
- Container
- Foldout
- Box
- Padding
- Spacing
- Children Layout

Value Drawer

`AbeValueDrawer<TValue>` 用于：

根据 Property 的 Value Type 决定如何绘制。

这里的 Value 指的是 Property 实际保存的数据类型。

例如：

`public Vector3 Position;`

它的 Value Type 是：

Vector3

因此可以创建：

```
public class Vector3Drawer
    : AbeValueDrawer<Vector3>
{
}
```

这个 Drawer 的匹配依据是：

Property Value Type == Vector3

而不是某个 Attribute。

Why Value Drawer Exists

Value Drawer 适用于：

某种数据类型本身就需要特殊的 Inspector GUI。

例如：

Vector2

Vector3

Color

AnimationCurve

Custom Struct

Custom Class

Enum

特殊数据类型

例如：

```
public struct TimeRange
{
    public int Start;
    public int End;
}
```

如果希望所有 TimeRange 都使用统一的 GUI：

```
public class TimeRangeDrawer
    : AbeValueDrawer<TimeRange>
```

```
{  
}
```

那么：

```
public TimeRange StartTime;
```

```
public TimeRange EndTime;
```

都可以使用这个 Drawer。

这里没有必要增加 Attribute。

因为：

改变绘制方式的原因就是它的 Value Type 是 TimeRange。

Attribute Drawer vs Value Drawer

这是两个非常容易混淆的概念。

例如：

```
[ProgressBar]
```

```
public int Health;
```

```
public int Damage;
```

如果：

Health

显示成 Progress Bar, 而：

Damage

保持普通 IntField, 那么原因是：

Health

↓

ProgressBarAttribute

↓

Attribute Drawer

而不是：

int

↓

Value Drawer<int>

因为如果使用：

```
AbeValueDrawer<int>
```

那么所有匹配的 `int Property` 都可能受到影响。

反过来：

```
public Vector3 Position;
```

```
public Vector3 Rotation;
```

如果你希望所有 `Vector3` 都使用特殊绘制：

```
Vector3
```

↓

```
Value Drawer<Vector3>
```

这时候就不需要 `Attribute`。

Choosing a Drawer

创建 `Drawer` 时，可以先问：

这个绘制行为为什么存在？

如果：

因为 `Property` 拥有某个 `Attribute`

使用：

```
AbeAttributeDrawer<TAttribute>
```

如果：

因为 `Property` 属于某个 `Group`

使用：

```
AbeGroupDrawer<TAttribute>
```

如果：

因为 `Property` 的 `Value Type` 是某种类型

使用：

```
AbeValueDrawer<TValue>
```

可以简单记成：

Attribute

→ Attribute Drawer

Group

→ Group Drawer

Value Type

→ Value Drawer

Drawer Locator

DrawerLocator 负责寻找 Property 对应的 Drawer。

它会根据 Property 当前的信息寻找匹配项。

例如：

Property

```
|
|— Attributes
|— Group
|— Value Type
```

▼

DrawerLocator

```
|
|— Attribute Drawer
|— Group Drawer
|— Value Drawer
```

找到的 Drawer 会进一步组成 Drawer Chain。

Drawer Chain

一个 Property 可以同时匹配多个 Drawer。

例如：

`[SomeAttribute]`

`public MyValue Value;`

可能同时满足：

`SomeAttribute`

↓

`Attribute Drawer`

`MyValue`

↓

`Value Drawer<MyValue>`

这些 `Drawer` 可以组合成：

`Attribute Drawer`

↓

`Value Drawer`

↓

`Default Drawing`

`Drawer Chain` 的职责是：

管理多个 `Drawer` 的执行顺序以及它们之间的调用关系。

因此：

`One Property`

↓

`Multiple Matching Drawers`

↓

`Drawer Chain`

而不是：

`One Property`

↓

`One Drawer`

`Default Drawing`

如果 `Property` 没有特殊 `Drawer`：

```
public int Health;
```

最终会进入默认绘制。

例如：

```
Health
```

```
[ 100 ]
```

因此完整流程可以是：

```
Property
```

↓

```
Drawer Locator
```

↓

```
Matching Drawers
```

↓

```
Drawer Chain
```

↓

```
Custom Drawer
```

↓

```
Default Drawing
```

Creating a New Drawer

创建新的 Drawer 通常需要：

1. 确定 Drawer 类型
2. 继承对应的 Drawer Base Class
3. 实现绘制逻辑

Attribute Drawer

首先定义 Attribute：

```
public class MyAttribute : Attribute
{
}
}
```

然后：

```
public class MyDrawer
```

```
        : AbeAttributeDrawer<MyAttribute>
    {
    }
```

使用:

```
[My]
public int Value;
```

流程:

```
MyAttribute
    ↓
DrawerLocator
    ↓
MyDrawer
    ↓
DrawerChain
    ↓
GUI
```

Group Drawer

定义 Group Attribute:

```
public class MyGroupAttribute : Attribute
{
}
```

然后:

```
public class MyGroupDrawer
    : AbeGroupDrawer<MyGroupAttribute>
{
}
```

用于处理整个 Group 的 GUI。

Value Drawer

定义自己的 Value Type:

```
public struct MyValue
{
    public int A;
    public int B;
}
```

然后:

```
public class MyValueDrawer
    : ABeValueDrawer<MyValue>
{
}
}
```

任何使用:

```
public MyValue Value;
```

的 Property 都可以匹配这个 Drawer。

Complete Example

假设我们有:

```
[ProgressBar]
```

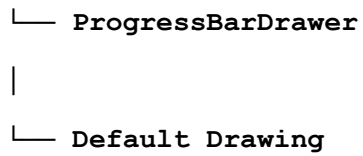
```
public int Health;
```

```
public Vector3 Position;
```

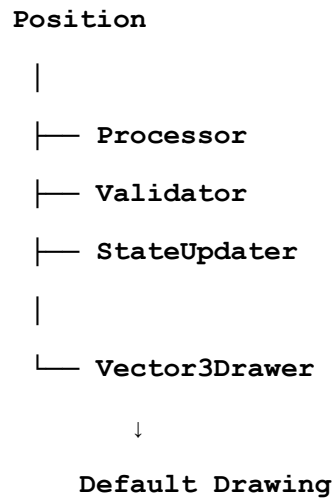
系统可能产生:

Health

```
|
|
|— ProgressBarAttribute
|
|— Processor
|
|— Validator
|
|— StateUpdater
|
```



而:



两者的区别在于:

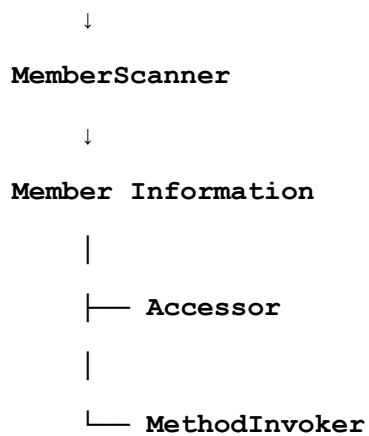
Health
→ Attribute 决定特殊绘制

Position
→ Value Type 决定特殊绘制

Reflection Pipeline

AbeAttributes 使用 Scanner、Accessor 和 Invoker 将 Reflection 操作集中处理。

TypeScanner



例如：

`"Settings.IsEnabled"`

↓

`Member Resolution`

↓

`Accessor / MethodInvoker`

↓

`Get Value`

Resolver 支持 Member Path 和 Parameterless Method。

解析后的 Delegate 会被缓存，以减少重复 Reflection 操作。

Current Features

当前 Core 包含：

- `AbeInspector`
- `AbePropertyTree`
- `AbeProperty`
- `TypeScanner`
- `MemberScanner`
- `Accessor`
- `MethodInvoker`
- `Attribute Resolution`
- `Property Processors`
- `Property Validators`
- `State Updaters`
- `Drawer Locator`
- `Drawer Chain`
- `Attribute Drawers`
- `Group Drawers`
- `Value Drawers`
- `Root-level Group Processing`

- Lazy SerializedProperty Children
 - Cached Reflection Delegates
 - Unity Serialization
-

Current Limitations

Group Processing

当前 Group Processing 主要针对 Root-level Properties。

Nested Group 尚未作为完整的通用 Group Hierarchy 处理。

Property Children

SerializedProperty Children 使用 Lazy Creation。

只有访问 Child 时才建立对应的 AbeProperty。

ShowIf

当前 ShowIf 的状态逻辑通过：

`PropertyUtility.IsVisible()`

进行处理。

Member Resolver

当前 Resolver 支持：

- Member Path
- Field
- Property
- Parameterless Method

并使用缓存 Delegate。

Expression

当前不实现：

`@expression`

语法。

系统目前使用 Member Path 和 Method Resolver，而不是完整的 C# Expression Parser。

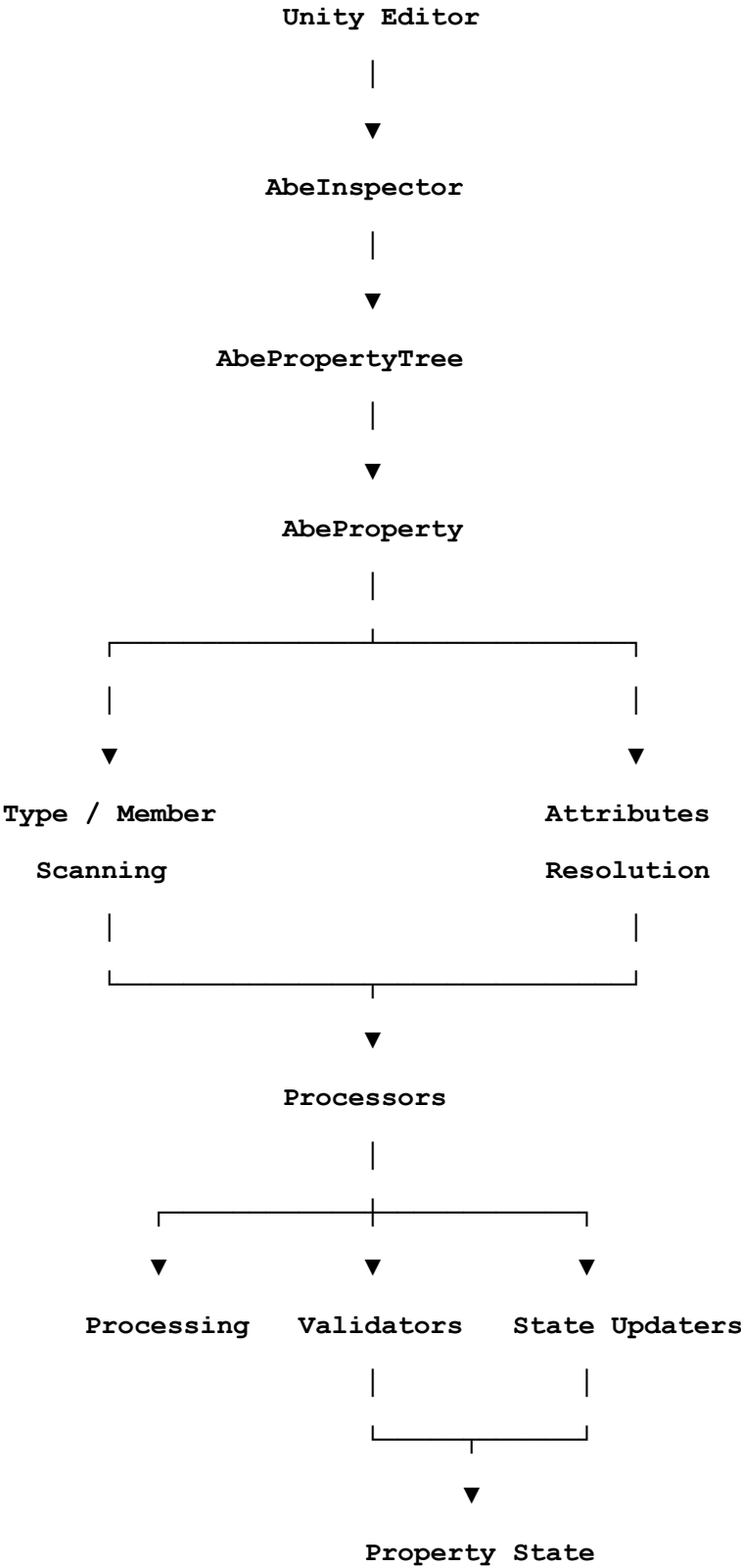
Serialization

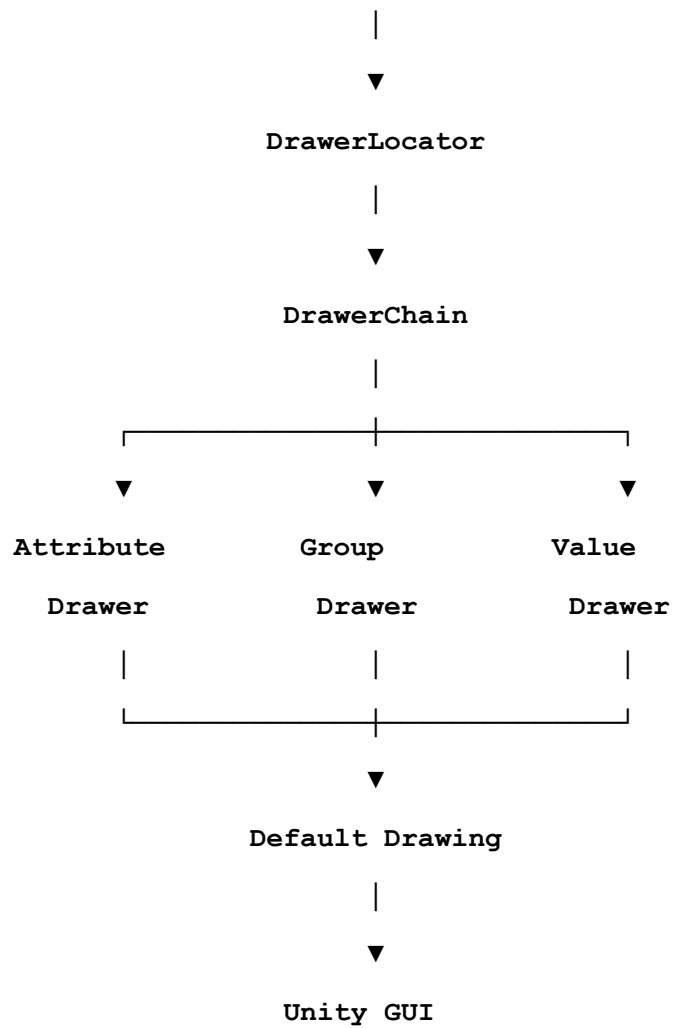
AbeAttributes 使用 Unity Serialization。

它本身不提供独立的 `Serialization System`。

Complete Pipeline

最终，整个 `AbeAttributes Inspector` 可以概括为：





核心职责可以浓缩成：

Scanner

→ 找到 Type / Member

Accessor

→ 访问 Value

MethodInvoker

→ 调用 Method

Processor

→ 处理 Property

Validator

→ 检查 Property / Value

StateUpdater

→ 更新 Property State

DrawerLocator

→ 找到 Drawer

DrawerChain

→ 组织 Drawer

Drawer

→ 绘制 GUI

这就是 AbeAttributes 的完整 Inspector Pipeline。